

Saabaco
Proyecto Integrador 2

Universidad ORT

2025

Índice general

1. Index	3
1.1. Saabaco	3
1.1.1. Descripción	3
2. Funcionamiento	5
2.0.1. Uso del sistema	5
3. Solución y diseño	6
4. ThingsBoard	15
5. Plan de trabajo	22
6. Bitácora	34
6.0.1. Martes 18/8	34
6.0.2. Jueves 21/8	34
6.0.3. Martes 26/8	34
6.0.4. Jueves 28/8	34
6.0.5. Martes 2/9	34
6.0.6. Jueves 4/9	34
6.0.7. Martes 9/9	34
6.0.8. Martes 16/9	35
6.0.9. Jueves 18/9	35
6.0.10. Martes 23/9	35
6.0.11. Viernes 26/9	35
6.0.12. Martes 30/9	35
6.0.13. Jueves 2/10	35
6.0.14. Martes 7/10	36
6.0.15. Jueves 9/10	36
6.0.16. Martes 21/10	36
6.0.17. Jueves 23/10	36
6.0.18. Martes 28/10	36
6.0.19. Jueves 30/10	36
6.0.20. Martes 4/11	36
6.0.21. Jueves 6/11	36
6.0.22. Martes 11/11	36
6.0.23. Jueves 13/11	37
6.0.24. Martes 18/11	37
6.0.25. Jueves 20/11	37

6.0.26. Martes 25/11	37
6.0.27. Jueves 27/11	37
A. Códigos de ambas placas	38
A.1. Código de la placa principal (ESP32)	38
A.2. Código de la placa secundaria (ESP8266)	48

Capítulo 1

Index

1.1. Saabaco

Este trabajo se realizó en el contexto de la materia Proyecto Integrador 2 - Universidad ORT 2025, el mismo se desarrolló utilizando github y github pages dentro de un proyecto privado, este documento es una recopilación de los puntos centrales de la documentación, con contenidos técnicos de la solución y de la gestión del proyecto.

1.1.1. Descripción

Problema a resolver

Saabaco (sistema automatizado de abastecimiento de combustibles) consiste en el desarrollo de un surtidor de autoservicio orientado a clientes regulares, como empresas de transporte, taxistas y compañías de reparto. Estos usuarios presentan una frecuencia de carga significativamente mayor que el consumidor común, lo que abre la oportunidad de optimizar el servicio, reduciendo tiempos de abastecimiento y mejorando la gestión administrativa asociada.

Para abordar esta necesidad se diseñó un surtidor controlado desde la nube, con mecanismos de pago integrados y medidas de seguridad ante situaciones de riesgo. El proyecto busca mejorar la eficiencia del proceso de carga, ofrecer un mejor servicio tanto a la empresa como al usuario, y garantizar un uso seguro y trazable de los recursos físicos y humanos involucrados.

Objetivo

El objetivo del proyecto, desde la función del prototipo, es ofrecer a quien presta el servicio de abastecimiento una herramienta para gestionar integralmente su surtidor a través de dashboards IoT. El sistema permite supervisar las reservas de combustible disponibles, administrar la base de clientes, visualizar cómo y cuándo consumen, y disponer de información para definir cómo cobrarles. Además, incorpora mecanismos de seguridad que, ante una condición de riesgo (por ejemplo, detección de humo), generan alertas de incendio y bloquean automáticamente el surtidor, deteniendo la bomba y evitando usos inseguros.

Desde el punto de vista del cliente final, el prototipo implementa un esquema de autoservicio supervisado: cada usuario se registra en una base de datos y se le asigna una tarjeta RFID con la que puede acercarse al surtidor, identificarse y realizar la carga sin

necesidad de pagar en el momento. De esta forma, empresas con flotas pueden controlar el consumo de sus empleados, ya que el sistema vincula cada operación a un usuario y cuenta con controles de consumo y límites configurables, ofreciendo una experiencia de carga sencilla en el surtidor y, al mismo tiempo, una gestión centralizada y trazable para el dueño del servicio.

Objetivos de aprendizaje

- Incorporar conceptos y prácticas de **IoT**, aprendiendo a vincular un sistema físico con una plataforma en la nube mediante protocolos como MQTT y el uso de dashboards y rule chains en ThingsBoard.
- Desarrollar la capacidad de **integrar múltiples elementos electrónicos** (sensors de nivel, caudal, humo, RFID, display LCD, bomba, fuentes de alimentación, etc.) en un mismo sistema, logrando que todos sean compatibles con el esquema de comunicación y control remoto.
- Diseñar la **arquitectura global del sistema** dividiéndola en etapas claras (hidráulica, electrónica y lógica en la nube), priorizando eficiencia, robustez y seguridad, y evitando complejidades innecesarias tanto en el hardware como en el software.
- Desarrollar la **gestión del proyecto y la coordinación entre áreas**, trabajando con pruebas de concepto y prototipos incrementales de manera que los desarrollos del sistema físico y los de la plataforma en la nube avancen de forma coherente, complementaria e iterativa, evitando bloqueos y permitiendo una integración progresiva de las funcionalidades

Capítulo 2

Funcionamiento

2.0.1. Uso del sistema

Cada nuevo usuario se registra en la plataforma y obtiene una tarjeta RFID asociada a su cuenta. En el momento del alta se configura, entre otros datos, un límite de volumen por operación, que restringe la cantidad máxima que puede consumir en cada compra.

Una vez dado de alta, el usuario puede acercarse al surtidor y seleccionar, mediante los botones del panel, el volumen que desea despachar. Cuando confirma su elección, se identifica acercando su tarjeta al lector; el sistema envía la solicitud a la nube y, si la petición es válida para ese cliente (usuario habilitado y volumen dentro de los límites configurados), la carga queda autorizada.

Con la operación habilitada, el usuario puede iniciar el abastecimiento presionando el botón correspondiente, que habilita el bombeo y le da control directo sobre el inicio de la carga. El sistema mide el volumen despachado y, una vez alcanzada la cantidad seleccionada, detiene automáticamente la bomba, finaliza la operación y el usuario puede retirarse.

Los usuarios no habilitados, así como aquellos que soliciten volúmenes fuera de los rangos permitidos, no podrán completar la operación y serán informados de la situación en la interfaz del surtidor. En caso de detectarse una condición de incendio, el sistema se bloquea inmediatamente, corta el abastecimiento y advierte al usuario. El surtidor permanece inhabilitado hasta que, una vez resuelta la incidencia, se reciba desde la nube la orden de reactivar el servicio.

Capítulo 3

Solución y diseño

El sistema físico se constituye en dos secciones complementarias, una etapa encargada del pasaje del fluido y un sistema electrónico encargado de censar, procesar y ejecutar según la lógica del sistema requiera.

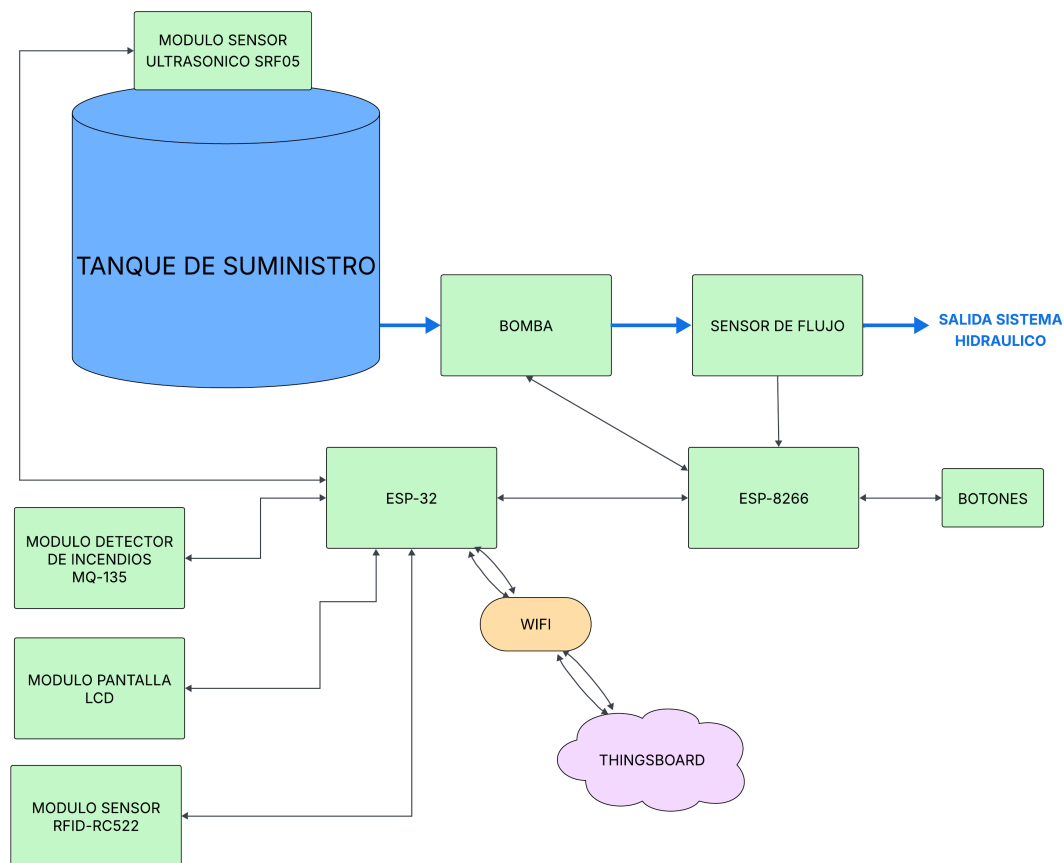


Figura 3.1: Diagrama del sistema.

La sección hidráulica comienza en un tanque de 4 litros, provisto de una tapa diseñada para sostener el sensor encargado de medir las reservas de combustible. Desde el tanque se desarrolla una red de tuberías en la que, al ponerse en marcha la bomba, el fluido es impulsado y la línea queda presurizada, forzando su paso a través del sensor de caudal y conduciéndolo finalmente por el tramo terminal de la línea, donde se realiza el abastecimiento al cliente.

Por otra parte, la sección electrónica se compone fundamentalmente de dos placas de desarrollo: la principal, una ESP32, y la secundaria, una ESP8266. El uso de dos placas permite distribuir las funciones implementadas en cada una, obteniendo así un sistema más robusto y de menor complejidad.

La placa secundaria se encarga de gestionar acciones bloqueantes que requieren alta precisión durante períodos prolongados, como el control de la bomba con módulo MOSFET, la medición precisa del flujo y la detección de las entradas del usuario mediante botones. La placa principal, en cambio, maneja la conexión bidireccional con la nube, manteniendo activa la comunicación y gestionando mensajes MQTT en ambos sentidos. Además, controla el resto de los sensores y actuadores del sistema, tales como el display LCD, el sensor RFID RC522, el sensor de humo MQ-135 y el sensor ultrasónico SRF05.

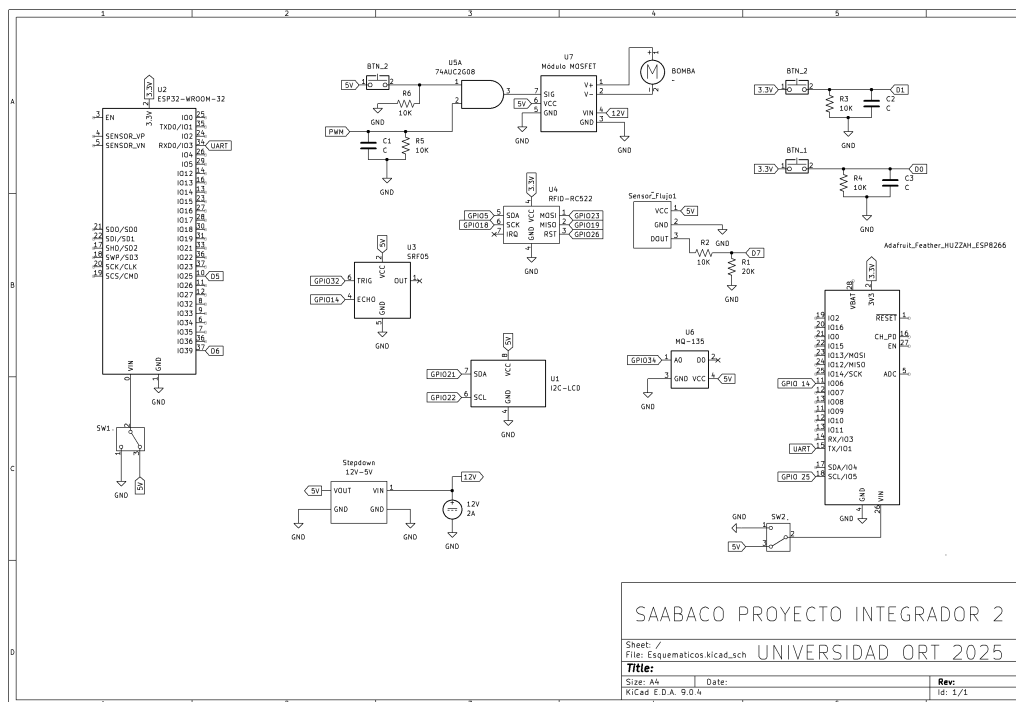


Figura 3.2: Diagrama esquemático de la sección electrónica del sistema.

Dada la funcionalidad de la placa principal, es necesario utilizar una lógica no bloqueante, ya que la conexión con la nube debe permanecer activa para enviar y recibir correctamente las telemetrías generadas. Este condicionamiento derivó en el diseño de una finite state machine (FSM), que, en conjunto con técnicas de programación adecuadas, permite que el sistema ejecute de forma iterativa un bloque de código crítico para la recepción de datos y el mantenimiento de la conexión con la nube.

Paralelamente, en la placa secundaria se implementó otra FSM, permitiendo implementar un lazo retroalimentado donde el funcionamiento de la bomba se controla según las mediciones del sensor de flujo. El accionar de la bomba también depende del usuario, la salida del PWM de la placa se compara analógicamente con una señal de enable manejada por el usuario con un pulsador.

Además, ambas placas se comunican mediante UART y pines de control, por lo que la correcta operación del sistema depende de la sincronización entre ellas, tarea que se

ve facilitada por el uso de las FSM implementadas, simplificando la coordinación entre ambos dispositivos.

Placa principal

En la placa principal, la máquina de estados comienza en un estado de inicio, donde se reinician las variables del ciclo y, a continuación, pasa al estado de espera de usuario. En este estado se recibe desde la placa secundaria la cantidad seleccionada, se muestra en la interfaz y se espera la identificación del usuario por RFID. Cuando el usuario se identifica, se envía una solicitud de habilitación a la nube y se entra en el estado de espera de respuesta. Si la respuesta indica usuario no habilitado, se descarta la operación y se regresa a la espera de usuario; si el usuario está habilitado, se avanza al estado de despacho, se ordena el inicio de bombeo a la placa secundaria y se permanece allí hasta recibir la señal de fin de abastecimiento, tras lo cual se vuelve a la espera de usuario. En cualquier momento, la detección de un evento de incendio provoca el paso al estado de alarma, donde se aborta el abastecimiento si está en curso y se mantiene la condición de alarma hasta que llega desde la nube la orden de desactivarla, momento en el que se retorna al estado de inicio.

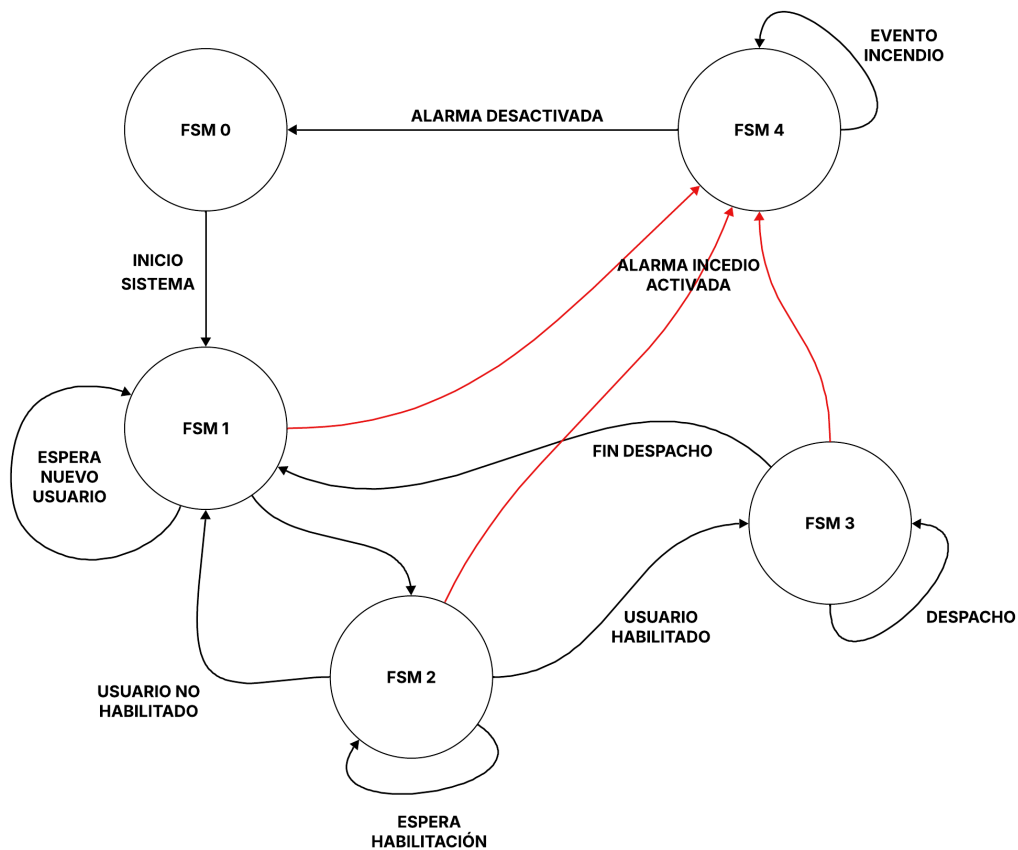


Figura 3.3: Diagrama de flujo FSM principal

```

1 switch(fsm)
2 {
3     case 0:
4         // Reinicio del sistema
  
```

```

5      estado_habilitacion = 0;
6      cantidad = 0;
7      alarma = true;
8      fsm = 1;
9      break;
10
11  case 1:
12
13
14      // Leer UART por cantidad
15      if(Serial.available() )
16      {
17          String recibirMensaje = Serial.readStringUntil('\n');
18          cantidad = recibirMensaje.toInt();
19      }
20
21      // Mostrar en LCD
22      mensajeCant = "          " +String(cantidad);
23      mensajePrecio = "          $" + String(precio);
24
25      static unsigned long lastLCD = 0;
26      if (millis() - lastLCD > 100) { // cada 0.1 s
27          clearDisplay();
28          mostrarLCD(msgCantidad, msgId , mensajeCant, mensajePrecio);
29          lastLCD = millis();
30      }
31
32
33      // Si hay una nueva lectura RFID, enviar telemetrias y cambiar a
estado 2
34      if(readRFID())
35      {
36          enviarTelemtria("cantidad_carga", String(cantidad));
37          enviarTelemtria("id_tarjeta", uidString);
38
39          fsm = 2;
40      }
41
42      break;
43
44  case 2:
45      // Espero RPC
46      // Si es 1(true), envio señal de control PWM actualizo LCD y cambio
a estado 3
47      // Si es -1(false), nvio mensaje LCD y vuelvo a estado 1
48      // Si es 0 es indeterminado
49      if(estados_habilitacion != 0)
50      {
51          if(estados_habilitacion == 1)
52          {
53              // Habilitado

```

```

54         conteoActivo = false;
55         digitalWrite(enablePwm, HIGH); // Habilito PWM
56         fsm = 3;
57         clearDisplay();
58         mostrarLCD("", "Despachando...", "", "");
59         estado_habilitacion = 0;
60         break;
61     }
62     else
63     {
64         // No habilitado
65         clearDisplay();
66         mostrarLCD("", "Usuario no", "habilitado", "");
67
68         if (!conteoActivo)
69         {
70             lastMsg2 = millis();
71             conteoActivo = true;
72         }
73
74         unsigned long now2 = millis();
75         if (now2 - lastMsg2 > 2000) // Muestro mensaje por 2 segundos
76         {
77             fsm = 1;
78             conteoActivo = false;
79             estado_habilitacion = 0;
80         }
81         break;
82     }
83 }
84
85 break;
86
87 case 3:
88     // Leo señal de control recibida
89
90     if(digitalRead(finBombeo) == HIGH) // Fin de bombeo
91     {
92         digitalWrite(enablePwm, LOW); // Apago PWM
93         clearDisplay();
94         mostrarLCD("", "Fin de bombeo", "", "");
95         fsm = 1;
96
97     }
98
99     // HIGH, LCD fin, señal control PWM LOW y state 1
100    // LOW, mantengo estado
101    break;
102
103 case 4:
104     // Alarma de incendio

```

```

105     enviarTelemtria("alarmaActiva", "true");
106     delay(1000); // Evitamos bombardear con telemetria
107     // Actualizo LCD
108     clearDisplay();
109     mostrarLCD("!!!ALARMA DE","INCENDIO ACTIVADA!!!","Apague la alarma"
110     ,"desde la app");
111
112     // Bloqueo PWM
113     digitalWrite(enablePwm, LOW);
114
115     // Implementar como volver a normalidad
116     if (!alarma) //Se desactivo la alarma
117     {
118         fsm = 0;
119     }
120     //El delay de este bloque esta implicito a recibir el RPC en t > 20
121     ms
    break;
}

```

Placa secundaria

En este caso la máquina de estados comienza en un estado de inicio del sistema, desde el cual pasa al estado de selección de cantidad una vez que ocurre la primera ejecución del sistema. En esta etapa el usuario ajusta el volumen deseado mediante los botones, mientras la placa lee y comunica continuamente dicho valor a la placa principal. Cuando desde la placa principal llega la orden lógica de compra realizada (autorización de despacho), la FSM avanza al estado de control de bombeo, donde se acciona la bomba y se supervisa el flujo hasta completar el volumen previamente seleccionado por el usuario, o en caso de que la placa principal aborte el bombeo. Ante la condición de bombeo finalizado o interrumpido, se genera la señal de fin de servicio hacia la placa principal y la FSM vuelve al estado de selección de cantidad.

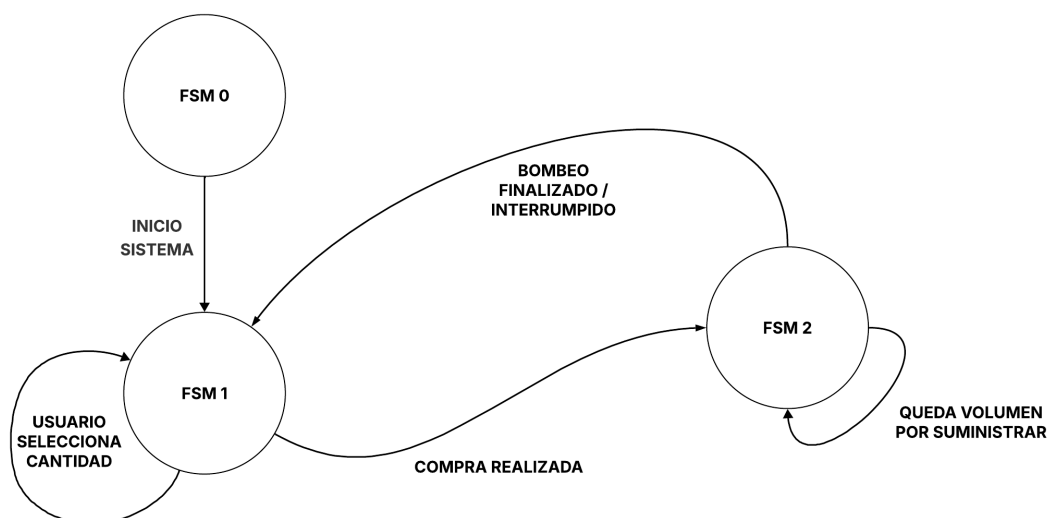


Figura 3.4: Diagrama de flujo FSM secundaria

```

1  switch (fsm) {
2
3      case 0: // Estado de inicio
4
5          fsm = 1;
6          break;
7
8      case 1: // Estado lectura de botones
9          digitalWrite(flagFin , LOW);
10         // Llego la señal para iniciar bombeo por el pin de control
11
12         if (digitalRead(flagBomba) == HIGH)
13         {
14             fsm = 2;
15             valorVol = cuenta;
16             cuenta = 0;
17             break;
18         }
19
20         if (digitalRead(btn1) == HIGH && cuenta < 2000)
21         {
22             cuenta = cuenta + 10;
23             delay(200);
24             mandarMensaje = String(cuenta);
25             Serial.println(mandarMensaje);
26
27
28         }
29
30         if (digitalRead(btn2) == HIGH && cuenta > 0)
31         {
32             cuenta = cuenta - 10;
33             delay(200);
34             mandarMensaje = String(cuenta);
35             Serial.println(mandarMensaje);
36
37
38         }
39
40
41
42         break;
43
44
45     case 2: // Estado control bomba
46
47         if (valorVol >= 5 && digitalRead(flagBomba) == HIGH) // valor ya
48         decrementado
49         {
50             // Generacion de señal PWM
51             unsigned long ahora = millis();

```

```

51     if (ahora - anterior >= medioPeriodo)
52     {
53         if (estado) {
54             digitalWrite(pwm, HIGH);
55         } else {
56             digitalWrite(pwm, LOW);
57         }
58         estado = !estado;
59         anterior = ahora;
60         break;
61     }
62
63     if (pulsos < 4)
64     {
65         if (digitalRead(flujo) == HIGH && !risingEdge)
66         {
67
68             risingEdge = true;
69             if (pulsos == 0)
70             {
71                 contador1 = millis();
72             }
73
74             unsigned long ahora = millis();
75             acumulado = acumulado + ahora - contador1;
76             pulsos = pulsos + 1;
77             contador1 = ahora;
78         }
79
80         if (digitalRead(flujo) == LOW && risingEdge)
81         {
82             risingEdge = false;
83         }
84
85         break;
86     }
87     else
88     {
89         float tiempoT = acumulado*0.001; // Escalado a segundos
90         float periodo = tiempoT / 3;
91         float caudal = 1.0 / (periodo*k); // Litros por segundo
92         valorVol = valorVol - tiempoT*caudal; // Volumen en litros
93
94         acumulado = 0;
95         pulsos = 0;
96         risingEdge = true; // Ignoro este pulso
97     }
98 }
99 else
100 {
101     digitalWrite(flagFin , HIGH); // Indico que el bombeo ha finalizado

```

```

102     mandarMensaje = String(cuenta);
103     Serial.println(mandarMensaje);
104
105
106     acumulado = 0;
107     pulsos = 0;
108     valorVol = 0;
109     fsm = 1;
110     delay(200); // La placa principal tiene un delay no bloqueante mayor
a 20 ms, este delay permite sincronizar la flagBomba
111     break;
112 }
113 break;
114 }

```

Alimentación:

Para la alimentación utilizamos una fuente conmutada de 12V 2A, con la que alimentamos directamente la bomba, y el resto del sistema por medio de un módulo conversor de 3A, con el que bajamos el voltaje a 5V. Particularmente el sensor RFID se alimenta con los 3.3V generados por la placa principal. Las alimentaciones de ambas placas cuentan con un switch individual cada una, permitiendo conectarlas a una computadora para ver el monitor serial de cada placa mientras se ejecuta el sistema.

Código placa principal y secundaria

A continuación se incluye el código completo de ambas placas.

- Código placa principal (ESP32)
- Código placa secundaria (ESP8266)

Capítulo 4

ThingsBoard

Descripción de IoT

Para la capa IoT del proyecto se utilizó ThingsBoard como plataforma de monitoreo, control y base de datos. ThingsBoard es un servicio en la nube para Internet of Things que permite recibir datos de los dispositivos, almacenarlos, procesarlos y mostrarlos en forma gráfica. En nuestro caso, la placa de control del surtidor se conecta por Wi-Fi a ThingsBoard y envía telemetría (estado del sistema, variables de proceso, eventos) y recibe órdenes desde la nube (habilitar carga, detener bomba, etc.).

Sobre esta plataforma se desarrolló un dashboard específico para Saabaco, donde el operador puede visualizar en tiempo real la información del surtidor. La lógica de funcionamiento se implementó mediante un rule chain, que es un flujo de nodos que define cómo se procesan los datos: validación de mensajes, generación de alarmas, registro de eventos y envío de comandos de vuelta a la placa. De esta forma, la integración con ThingsBoard convierte al sistema en una solución IoT completa, con supervisión remota y control centralizado del equipo de abastecimiento.

A continuación se estará mostrando la rule chain y dashboards del sistema.

En este panel es posible agregar nuevos usuarios, otorgándoles acceso a la carga en el surtidor. Una vez registrado, el usuario aparece en la tabla de la derecha, donde se muestra la información de cada uno y se actualiza en tiempo real.

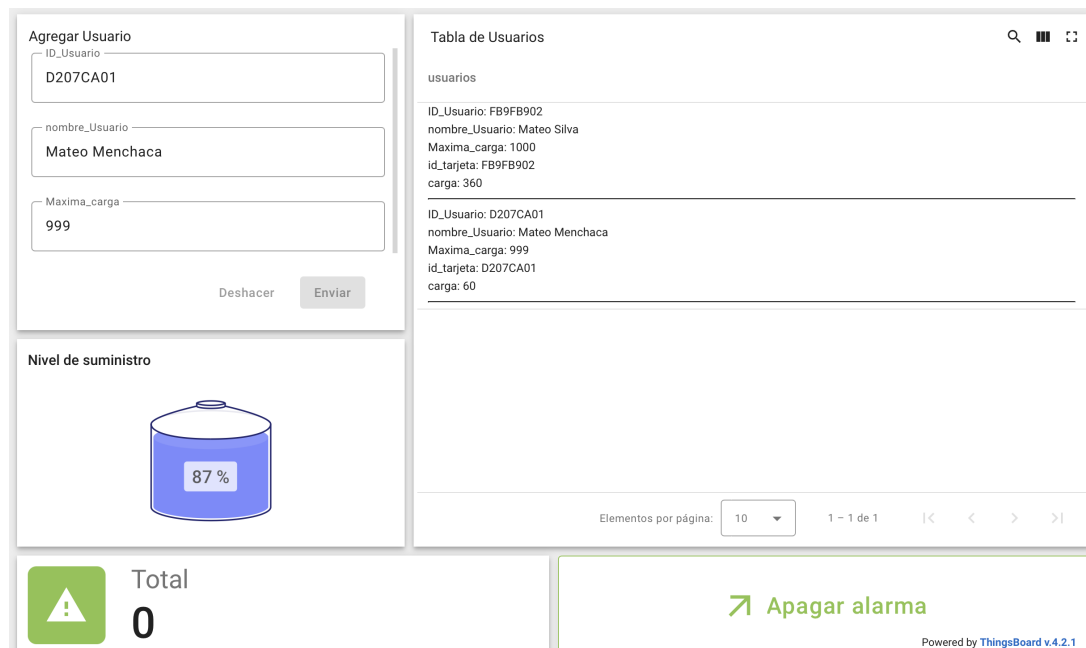


Figura 4.1: DashboardEmpresa

En este panel es posible agregar nuevos usuarios, otorgándoles acceso a la carga en el surtidor. Una vez registrado, el usuario aparece en la tabla de la derecha, donde se muestra la información de cada uno y se actualiza en tiempo real.

Además, se incluye un indicador de nivel de suministro del tanque, que funciona mediante telemetría y muestra la cantidad de combustible disponible. Finalmente, en la parte inferior se encuentra la opción de “apagar alarma”, que puede utilizarse cuando se genera una alarma. Esta se visualizará en el sector izquierdo del dashboard y disparará una notificación.

La tabla de este y el otro dashboard se ejecuta con el siguiente código, en la parte de editar las claves de datos de la misma:

```

1 var jsonObject = typeof value === "string" ? JSON.parse(value) : value;
2 var usuario = "";
3 // Iterar sobre cada clave
4 for (var ID in jsonObject) {
5     if (jsonObject.hasOwnProperty(ID)) {
6         var detalles = jsonObject[ID];
7
8         if (typeof detalles === "object") {
9             for (var atributo in detalles) {
10                 if (detalles.hasOwnProperty(atributo)) {
11                     usuario += atributo + ": " + detalles[atributo] + "<br>";
12                 }
13             }
14             usuario += "<hr>"; // raya
15         }
16     }
17 }
18
19 return usuario;

```

Este código recibe un objeto JSON que contiene la información de uno o varios usuarios.

A continuación, recorre cada entrada del objeto (identificada por un ID), para cada una, itera sobre sus atributos internos. En cada iteración construye una cadena de texto en formato HTML, concatenando pares atributo: valor separados por saltos de línea (
).

Al finalizar el procesamiento de cada usuario, agrega una línea divisoria (<hr>) para separar visualmente la información de un usuario de otro. Finalmente, la función devuelve esta cadena HTML completa, que luego se muestra en el dashboard para presentar de forma legible todos los datos del/los usuario/s.

Pasando ahora al dashboard del usuario, este panel, como indica su nombre, está pensado para que cada usuario pueda visualizar su propia información. A continuación se muestra su diseño:

Iniciar sesión	
ID_Sesion	D207CA01
Deshacer aceptar	

Datos de usuario	
sesion	
ID_Usuario:	D207CA01
nombre_Usuario:	Mateo Menchaca
Maxima_carga:	999
id_tarjeta:	D207CA01
carga:	60

Pay

Deuda

Última actualización hace 2s atrás

181 \$USD

Figura 4.2: DashboardUsuario

Este dashboard cuenta con un espacio para ingresar el usuario. A partir de la ID ingresada, si el usuario está registrado en la base de datos, su información aparece a la derecha, mostrando el usuario y todos sus datos asociados.

De igual manera, incluye un apartado para liquidar deudas, donde el monto se presenta convertido a dólares y, al presionar el botón “Pay”, se cancelan las deudas que tenga el usuario.

Finalmente, en la parte inferior se encuentra una gráfica que muestra las cargas realizadas por el usuario durante el último mes.

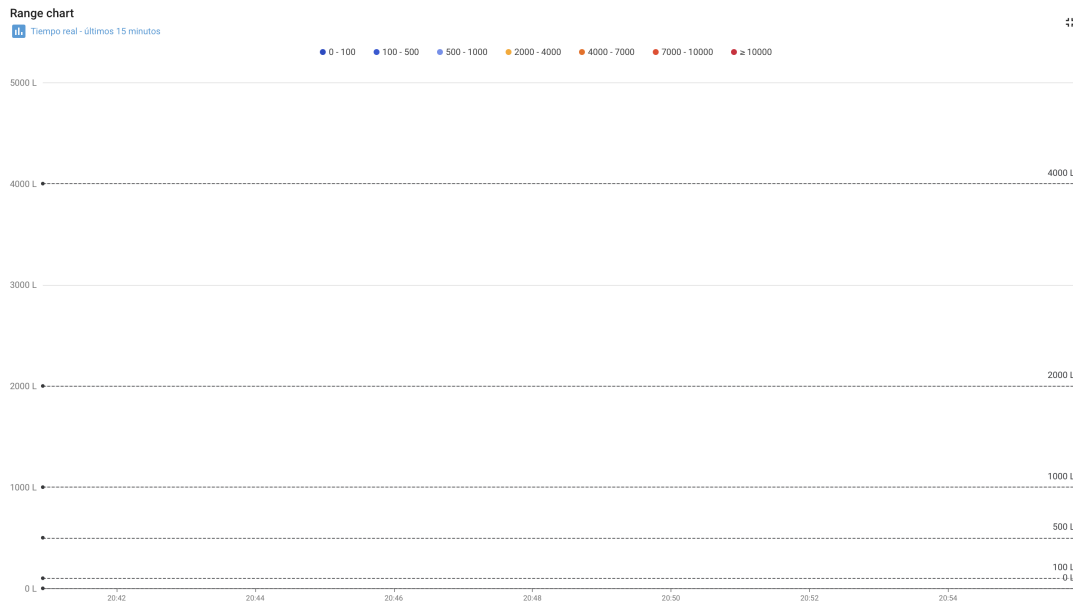


Figura 4.3: Grafica

Estos fueron los dashboards de Saabaco, el dashboard de usuario y el de administrador. A continuación se presentarán las rule chains, que son las encargadas de que estos dashboards funcionen de la manera deseada, procesando la información y gestionando la lógica del sistema.

Comenzando por la Root rule chain, esta consiste en la raíz de el funcionamiento del sistema, ya que cuando se realiza algo en el programa, primero pasa por esta rule chain la cual envia a donde corresponda cada mensaje, a la cual no realizamos muchos cambios, partiendo de la inicial, para evitar complicaciones por lo que se altero minimamente, sumando las rulechains hechas por Saabaco y aplicando algun delay en algun momento para evitar algun problema por mucho envio de datos.

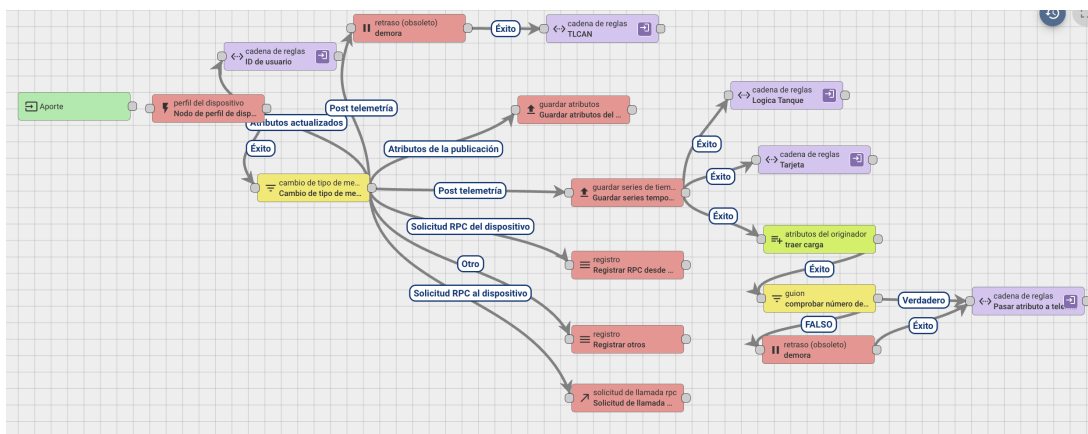


Figura 4.4: Root

Las entradas del dashboard se vinculan con las siguientes Rule Chains:

- **TLCAN**: lleva a la Rule Chain que ejecuta la API.
- **ID de usuario**: abre la Rule Chain de atributos, donde se leen y manejan los datos del usuario.

- **Tarjeta:** lleva a la Rule Chain de telemetría, donde se procesan los datos enviados por la placa.
- **Lógica tanque:** conecta con la Rule Chain que ejecuta la **actualización del nivel del tanque**.
- **Pasar atributo a telemetría:** corresponde a la Rule Chain de la gráfica, que toma atributos y los transforma en datos de telemetría para poder mostrarlos en el gráfico.

Una vez explicada la cadena raíz, vamos a comenzar por las que hacen que Saabaco funcione. La primera es la Rule Chain de telemetría, que se muestra a continuación:

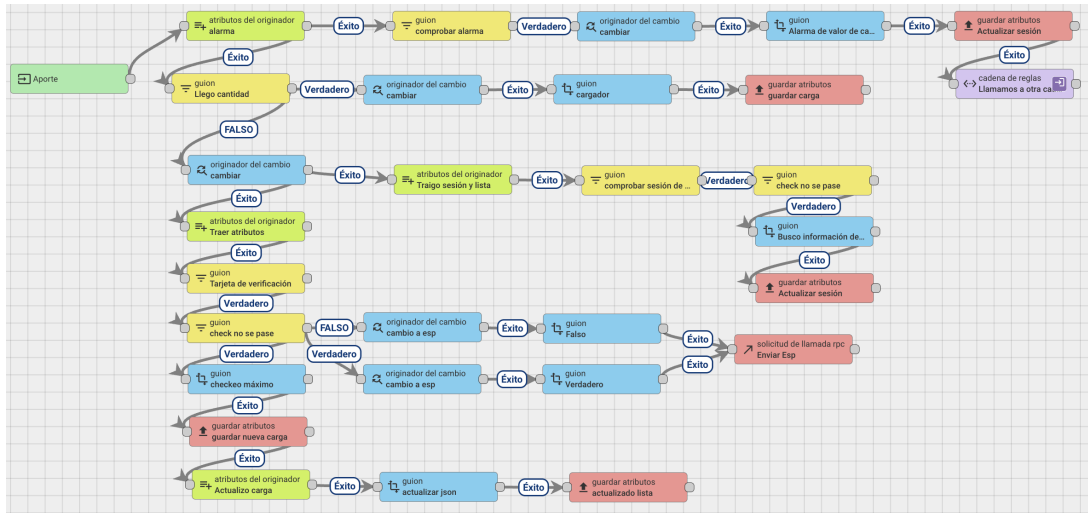


Figura 4.5: Telemetria

Esta Rule Chain comienza recogiendo los valores de la telemetría, para luego ir moviéndose por los distintos nodos. La primera línea se encarga de verificar si hay alguna alarma activa; en caso afirmativo, deriva a la Rule Chain de atributos para encender la alarma en el sistema.

A su vez, desciende y verifica la cantidad con la que se realizó una carga. Si esta condición es verdadera, pasa a la derecha y guarda el atributo en la base de datos de los usuarios, cambiando el originador al activo **Usuarios**, el cual contiene todos los atributos del sistema, por lo que se reutiliza en varias Rule Chains. Si la condición es falsa (es decir, no se quiere cargar), cambia el originador al activo correspondiente y pasa al nodo de la derecha, que crea una sesión, útil luego para la lógica de los dashboards. Finalmente, continúa descendiendo, donde chequea si se inició una sesión y si esta está registrada o no, enviando una señal RPC a la ESP. Si la sesión es válida, se registra la carga solicitada por el usuario.

Pasando a la lógica del tanque, cabe remarcar que podría haberse integrado en la parte de telemetría; sin embargo, se decidió implementar una Rule Chain aparte para evitar tener todo acumulado y muy desordenado en una sola cadena. A continuación se presenta la cadena de reglas que procesa la lógica del tanque:

<https://www.fueleconomy.gov/ws/rest/fuelprices>, con la cual calculamos los precios de la nafta en tiempo real.

A continuación se puede apreciar la Rule Chain que la maneja:

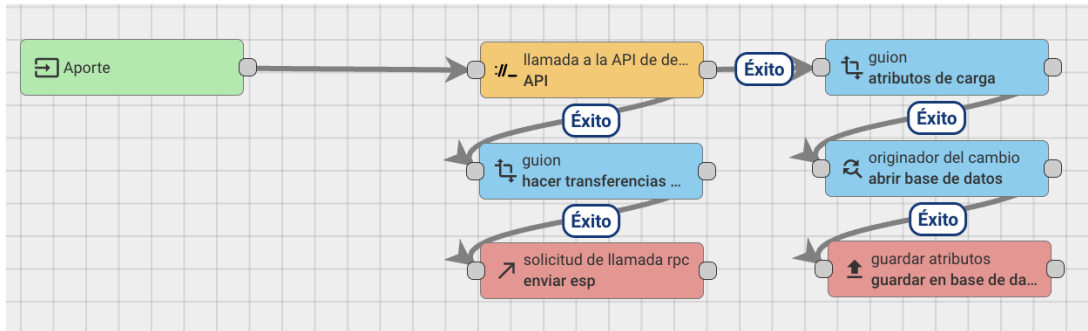


Figura 4.8: API

Cada vez que se ingresa en esta cadena se accede a la API, que devuelve los valores de la nafta en formato JSON. Luego se envía un mensaje RPC a la ESP y, a su vez, se crea un atributo para guardarlo en la base de datos de **Usuarios**.

Para culminar con las Rule Chains, presentamos una diseñada exclusivamente para la gráfica de las cargas de los usuarios, la cual se puede apreciar a continuación:

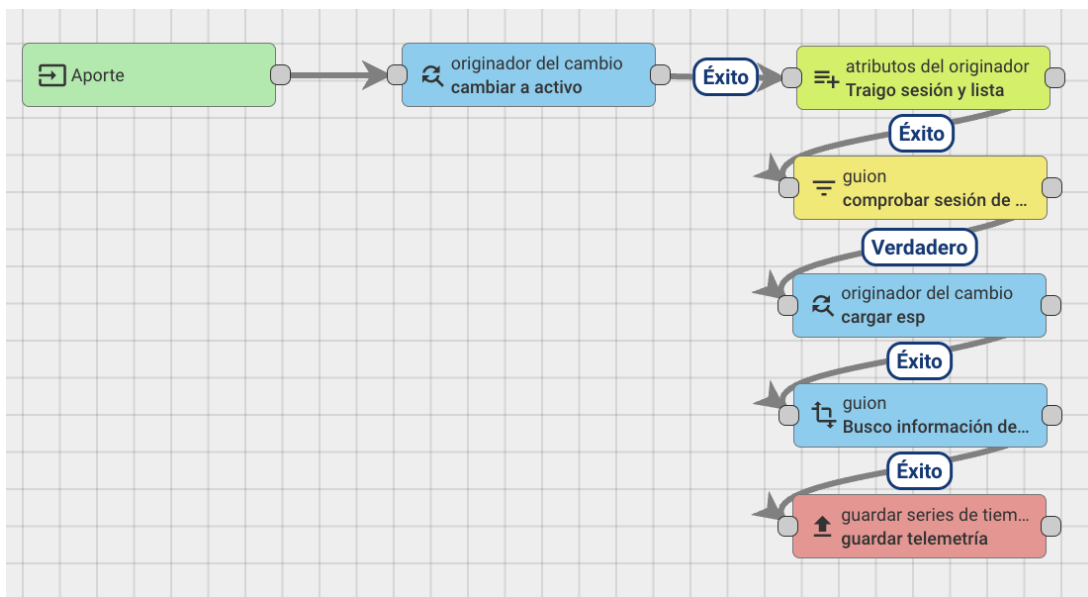


Figura 4.9: AtributoATElemetria

Esta Rule Chain se ejecuta cada vez que llega un mensaje por telemetría. Primero cambia a la base de datos, ya que se quiere obtener la sesión en la que estamos actualmente, y luego publica este dato como una telemetría para la ESP. Esto fue necesario porque la gráfica únicamente funciona con dispositivos y no con activos.

Capítulo 5

Plan de trabajo

Plan de trabajo del proyecto Saabaco

El plan de trabajo de Saabaco se estructuró en torno a dos ejes principales: una fase de pruebas de concepto y una fase de prototipos, desarrolladas de manera conjunta e iterativa. A medida que se realizaban dos o tres pruebas de concepto sobre los distintos módulos (sensores, comunicación, lógica de control), se iban incorporando esos resultados en prototipos cada vez más completos, hasta llegar al surtidor funcional actual.

Pruebas de concepto En esta primera etapa se realizaron ensayos unitarios para detectar problemas tempranos y ajustar el diseño:

- Verificación del **sensor ultrasónico** para medir el nivel del tanque y la proximidad dentro del sistema.

En esta prueba de concepto se utilizó el siguiente código para obtener, a partir de las señales del sensor, la distancia al objeto enfrenteado al mismo. De este modo se determinó el rango útil y la precisión de medida, pensando en que, una vez instalado en la parte superior del tanque, la variación de esta distancia permitirá inferir el nivel del líquido en el tanque.

```
1  #include <Arduino.h>
2  // Definir los pines del Arduino
3  const int trigPin = 9; // Pin Trigger
4  const int echoPin = 10; // Pin Echo
5
6  // Variables para la duración y la distancia
7  long duracion;
8  long distancia;
9
10 void setup() {
11     // Inicializar el monitor serial para mostrar la distancia
12     Serial.begin(115200);
13
14     // Configurar el pin Trigger como salida
15     pinMode(trigPin, OUTPUT);
16     // Configurar el pin Echo como entrada
17     pinMode(echoPin, INPUT);
18 }
19
```

```

20 void loop() {
21     // 1. Enviar un pulso de 10 microsegundos al pin Trigger para iniciar la
        medición
22     digitalWrite(trigPin, LOW);
23     delayMicroseconds(2); // Un pequeño delay antes de enviar el pulso alto
24     digitalWrite(trigPin, HIGH);
25     delayMicroseconds(10);
26     digitalWrite(trigPin, LOW);
27
28     // 2. Medir el tiempo de retorno del pulso en el pin Echo
29     // La función pulseIn() retorna el tiempo que el pin estuvo en estado
        HIGH
30     duracion = pulseIn(echoPin, HIGH);
31
32     // 3. Calcular la distancia en centímetros
33     // La fórmula es: distancia = (duración * velocidad_del_sonido) / 2
34     // La velocidad del sonido es aproximadamente 340 m/s o 29 microsegundos
        por centímetro.
35     // Dividimos entre 2 porque el tiempo es de ida y vuelta (hasta el
        objeto y de regreso).
36     distancia = duracion / 29 / 2;
37     distancia = int(distancia);
38     // 4. Mostrar la distancia en el Monitor Serial
39     Serial.print("Distancia: ");
40     Serial.print(distancia);
41     Serial.println(" cm");
42
43     // 5. Esperar antes de la siguiente medición (recomendado para evitar
        interferencias)
44     delay(1000); // Esperar 100 milisegundos
45 }

```

- Pruebas del **sensor de flujo y bomba**, garantizando una entrega controlada de caudal según la señal de entrada.

Se desarrolló un código para estimar el caudal a partir de la frecuencia de la señal del sensor y, mediante una constante de calibración, obtener el volumen correspondiente al fluido que atraviesa el sensor en cada intervalo de medida. Para evaluar la integridad de la señal y descartar errores por ruido, se comparó la lectura original con la misma señal filtrada mediante un pasabajos. Si bien el filtrado mejoró levemente la forma de la señal, no produjo cambios significativos en la precisión del volumen medido, por lo que se decidió prescindir de él.

Además, el código se diseñó de forma no bloqueante, ya que debe ejecutarse en paralelo con la generación de PWM para la bomba. Para ello se definió un criterio de promediado sobre cuatro períodos de la señal por cada valor considerado, mejorando la precisión de la estimación evitando ponderar medidas no del todo correctas producto de las funciones ocurriendo en paralelo.

```

1 const int flujo = 15; // Entrada sensor flujo D8
2
3 // Sensor Flujo

```



```

4 bool risingEdge = false;
5 unsigned long contador1 = 0;
6 long acumulado = 0;
7 int pulsos = 0;
8 const float k = 7.1; // Constante de calibracion del sensor de flujo
9
10 // Generacion de señal PWM (no bloqueante)
11 unsigned long ahora = millis();
12 if (ahora - anterior >= medioPeriodo)
13 {
14     if (estado) {
15         digitalWrite(pwm, HIGH);
16     } else {
17         digitalWrite(pwm, LOW);
18     }
19     estado = !estado;
20     anterior = ahora;
21     break;
22 }
23
24 // Medición de flujo promediando 4 pulsos (3 períodos)
25 if (pulsos < 4)
26 {
27     if (digitalRead(flujo) == HIGH && !risingEdge)
28     {
29         risingEdge = true;
30         if (pulsos == 0)
31         {
32             contador1 = millis();
33         }
34
35         unsigned long ahora = millis();
36         acumulado = acumulado + ahora - contador1;
37         pulsos = pulsos + 1;
38         contador1 = ahora;
39     }
40
41     if (digitalRead(flujo) == LOW && risingEdge)
42     {
43         risingEdge = false;
44     }
45 }
46 else
47 {
48     float tiempoT = acumulado*0.001; // Escalado a segundos
49     float periodo = tiempoT / 3;
50     float caudal = 1.0 / (periodo*k); // Litros por segundo
51     valorVol = valorVol - tiempoT*caudal; // Volumen en litros
52
53     acumulado = 0;
54     pulsos = 0;

```

```

55     risingEdge = true; // Ignoro este pulso
56 }

```

- Implementación del **módulo RFID RC522** para la identificación y validación de tarjetas de usuario.

En esta prueba de concepto nos enfocamos en comprender el funcionamiento del módulo y sus limitaciones. Para ello, utilizamos un código que permite leer una tarjeta, obtener su identificador y validar una tarjeta específica a partir de dicho valor. Además, exploramos el comportamiento del módulo con otras tarjetas pasivas de uso cotidiano, lo que resultó útil como herramienta de depuración del sistema, ya que, al ser correctamente interpretadas por el módulo, permiten generar solicitudes de usuarios no habilitados y verificar el tratamiento de estos casos.

```

1  #include <SPI.h>//https://www.arduino.cc/en/reference/SPI
2  #include <MFRC522.h>//https://github.com/miguelbalboa/rfid
3  #include <Arduino.h>
4
5  // 3.3V
6  //Pines SPI de la esp32
7  #define SS_PIN 5 // SDA
8  #define RST_PIN 26 // Reset
9  #define LED_PIN 2
10 //Parameters
11 const int ipaddress[4] = {103, 97, 67, 25};
12
13 //Variables
14 byte nuidPICC[4] = {0, 0, 0, 0};
15 MFRC522::MIFARE_Key key;
16 MFRC522 rfid = MFRC522(SS_PIN, RST_PIN);
17
18 void setup() {
19     //Init Serial USB
20     Serial.begin(115200);
21     Serial.println(F("Initialize System"));
22     //init rfid D8,D5,D6,D7
23     SPI.begin();
24     rfid.PCD_Init();
25     pinMode(LED_PIN, OUTPUT);
26     digitalWrite(LED_PIN, LOW);
27     Serial.print(F("Reader :"));
28     rfid.PCD_DumpVersionToSerial();
29 }
30
31 // Helper routine to dump a byte array as dec values to Serial.
32
33 void printDec(byte *buffer, byte bufferSize) {
34     for (byte i = 0; i < bufferSize; i++) {
35         Serial.print(buffer[i] < 0x10 ? " 0" : " ");
36         Serial.print(buffer[i], DEC);
37     }
38 }

```

```

39
40
41 bool readRFID() {
42     if (!rfid.PICC_IsNewCardPresent()) return false; // no hay tarjeta
43     if (!rfid.PICC_ReadCardSerial()) return false;    // no se pudo leer UID
44
45     Serial.print(F("UID: "));
46     String uidString = "";
47     for (byte i = 0; i < rfid.uid.size; i++) {
48         if (rfid.uid.uidByte[i] < 0x10) uidString += "0";
49         uidString += String(rfid.uid.uidByte[i], HEX);
50     }
51     uidString.toUpperCase();
52
53     Serial.println("UID detectado: " + uidString);
54
55     rfid.PICC_HaltA();
56     return true;
57 }
58
59
60
61
62     //Helper routine to dump a byte array as hex values to Serial.
63
64 void printHex(byte *buffer, byte bufferSize) {
65     for (byte i = 0; i < bufferSize; i++) {
66         Serial.print(buffer[i] < 0x10 ? " 0" : " ");
67         Serial.print(buffer[i], HEX);
68     }
69 }
70
71
72
73 void loop() {
74     if (readRFID()) {
75         Serial.println(F("Lectura correcta"));
76         digitalWrite(LED_PIN, HIGH);
77     }
78     else {
79         digitalWrite(LED_PIN, LOW);
80     }
81 }

```

- Ensayo del **sensor MQ-135** para detección de humo o gases y activación de medidas de seguridad.

Para los ensayos con el sensor MQ-135 se utilizó un código sencillo que permitió visualizar las lecturas analógicas entregadas por el dispositivo, ajustando su potenciómetro para trabajar en el modo de máxima sensibilidad. En las primeras pruebas, exponiéndolo a una cantidad de humo controlada, se observó que incluso en su nivel

más sensible no se obtenían variaciones suficientemente claras como para detectar de forma confiable la presencia de humo.

Tras un período de investigación, se identificó que este tipo de sensores requiere un tiempo de burn-in proporcional al tiempo y a las condiciones de almacenamiento previas. Este proceso consiste en alimentar el sensor de forma continua durante unas 72 horas, permitiendo estabilizar las propiedades eléctricas del material activo. Una vez completada esta etapa de estabilización, fue posible definir un umbral de medición adecuado para detectar la presencia de humo de forma consistente y coherente con las condiciones reales de ensayo.

```
1
2 void setup() {
3   Serial.begin(115200);
4   analogReadResolution(12); // Lectura de 0-4095
5 }
6
7 void loop() {
8   int adc_MQ = analogRead(34);
9   float voltaje = adc_MQ * (5 / 4095.0); // Conversión a voltaje
10
11   Serial.print("adc:");
12   Serial.print(adc_MQ);
13   Serial.print("    voltaje:");
14   Serial.println(voltaje);
15   delay(1000);
16 }
```

- Envío y recepción de datos con la nube mediante **ThingsBoard**, confirmando el funcionamiento de la telemetría y el monitoreo remoto.

Esta serie de pruebas de concepto tuvo como propósito inicial conectar la ESP32 a ThingsBoard y, una vez identificada, lograr generar telemetrías desde la placa y recibirlas correctamente en la nube. Cuando pudimos establecer la conexión de forma estable, realizamos una siguiente prueba donde incorporamos el sensor de distancia previamente probado para generar telemetría real. Con esto, nos propusimos enviar datos de forma periódica desde la placa e identificar cada mensaje desde una rule chain, corroborando el funcionamiento completo de la comunicación. Posteriormente, realizamos una última prueba manejando el flujo inverso: generamos un RPC desde la nube, que fue detectado por la placa y mostrado en el monitor serie.

```
1
2 #include <Arduino.h>
3 #include <WiFi.h>
4 #include <PubSubClient.h>
5 #include <ArduinoJson.h>
6
7 // Update these with values suitable for your network.
8 //const char* ssid = "HUAWEI-IoT";
9 //const char* password = "ORTWiFiIoT";
10 const char* ssid = "canelita";
11 const char* password = "Mostro1515";
```

```

12  const char* mqtt_server = "demo.thingsboard.io";
13  const char* token = "XigmXofElTIqmQUQHhJB";
14
15  //Sensor Distancia
16  const int trigPin = 9; // Pin Trigger
17  const int echoPin = 10; // Pin Echo
18
19  // Variables para la duración y la distancia
20  long duracion;
21  long distancia;
22
23  // Connection objects
24  WiFiClient espClient;
25  PubSubClient client(espClient);
26
27  //timestamp of the last telemetry update
28  unsigned long lastMsg = 0;
29
30  //buffer sizes and messages
31  #define MSG_BUFFER_SIZE (50)
32  char msg[MSG_BUFFER_SIZE];
33  char msg2[MSG_BUFFER_SIZE];
34
35  //json document to store incoming messages
36  DynamicJsonDocument incoming_message(256);
37
38  //Initiates WiFi Connection
39  void setup_wifi() {
40
41      delay(10);
42      // We start by connecting to a WiFi network
43      Serial.println();
44      Serial.print("Connecting to ");
45      Serial.println(ssid);
46
47      WiFi.mode(WIFI_STA);
48      WiFi.begin(ssid, password);
49
50      while (WiFi.status() != WL_CONNECTED) {
51          delay(500);
52          Serial.print(".");
53      }
54
55      randomSeed(micros());
56
57      Serial.println("");
58      Serial.println("WiFi connected");
59      Serial.println("IP address: ");
60      Serial.println(WiFi.localIP());
61  }
62

```

```

63 //This method is called whenever a MQTT message arrives. We must be
    prepared for any type of incoming message.
64 //We are subscribed to RPC Calls: v1/devices/me/rpc/request/+
65 void callback(char* topic, byte* payload, unsigned int length) {
66
67     //log to console
68     Serial.print("Message arrived [");
69     Serial.print(topic);
70     Serial.print("] ");
71     for (int i = 0; i < length; i++) {
72         Serial.print((char)payload[i]);
73     }
74     Serial.println();
75
76     //convert topic to string to parse
77     String _topic = String(topic);
78
79     if (_topic.startsWith("v1/devices/me/rpc/request/")) { //Aca se manejan
        las Request de thingsboard
80         //We are in a request, check request number
81         String _number = _topic.substring(26);
82
83         //Read JSON Object
84         deserializeJson(incoming_message, payload);
85         String metodo = incoming_message["method"];
86
87         if (metodo=="checkStatus") { //Check device status. Expects a
            response to the same topic number with status=true.
88             char outTopic[128];
89             ("v1/devices/me/rpc/response/"+_number).toCharArray(outTopic,128);
90
91             DynamicJsonDocument resp(256);
92             resp["status"] = true;
93             char buffer[256];
94             serializeJson(resp, buffer);
95             client.publish(outTopic, buffer);
96
97         }
98     }
99 }
100
101 void reconnect() {
102     // Loop until we're reconnected
103     while (!client.connected()) {
104         Serial.print("Attempting MQTT connection...");
105         // Attempt to connect - client.connect(DEVICE_ID, TOKEN, TOKEN)
106         if (client.connect("NODEMCU Nuevo", token, token )) {
107             Serial.println("connected");
108             // Once connected, subscribe to rpc topic
109             client.subscribe("v1/devices/me/rpc/request/+");
110         } else {

```

```

111     Serial.print("failed, rc=");
112     Serial.print(client.state());
113     Serial.println(" try again in 5 seconds");
114     // Wait 5 seconds before retrying
115     delay(5000);
116 }
117 }
118 }
119 // El nombre incluye EJ para diferenciarse del archivo main.cpp
120 int SensorDistanciaEJ()
121 {
122     // 1. Enviar un pulso de 10 microsegundos al pin Trigger para
123     iniciar la medición
124     digitalWrite(trigPin, LOW);
125     delayMicroseconds(2); // Un pequeño delay antes de enviar el pulso alto
126     digitalWrite(trigPin, HIGH);
127     delayMicroseconds(10);
128     digitalWrite(trigPin, LOW);
129
130     // 2. Medir el tiempo de retorno del pulso en el pin Echo
131     // La función pulseIn() retorna el tiempo que el pin estuvo en estado
132     HIGH
133     long duracion = pulseIn(echoPin, HIGH);
134
135     // 3. Calcular la distancia en centímetros
136
137     int distancia = duracion / 29 / 2;
138     distancia = int(distancia);
139
140     // 4. Mostrar la distancia en el Monitor Serial
141     Serial.print("Distancia: ");
142     Serial.print(distancia);
143     Serial.println(" cm");
144
145     return distancia;
146 }
147
148 void setup() {
149     //pinMode(BUILTIN_LED, OUTPUT); // Initialize the BUILTIN_LED pin
150     as an output
151     Serial.begin(115200);
152     setup_wifi();
153     client.setServer(mqtt_server, 1883);
154     client.setCallback(callback);
155
156     // Configurar el pin Trigger como salida sensor distancia
157     pinMode(trigPin, OUTPUT);
158     // Configurar el pin Echo como entrada sensor distancia
159     pinMode(echoPin, INPUT);
160 }

```

```

159
160 void loop() {
161
162     if (!client.connected()) {
163         reconnect();
164     }
165     client.loop();
166
167     unsigned long now = millis();
168     if (now - lastMsg > 2000) {
169         lastMsg = now;
170         int value = int(SensorDistanciaEJ());
171
172         //Send value as telemetry
173         DynamicJsonDocument resp(256);
174         resp["distancia_cm"] = value;
175         char buffer[256];
176         serializeJson(resp, buffer);
177
178         client.publish("v1/devices/me/telemetry", buffer);
179
180         Serial.print("Publish message [telemetry]: ");
181         Serial.println(buffer);
182     }
183 }

```

- Coordinación del **display**, asegurando una correcta visualización de la información a comunicar.

El display LCD incorpora un módulo de interfaz serie I2C, lo que simplifica considerablemente su control desde el microcontrolador. En esta prueba de concepto se utilizaron librerías específicas para, de forma conceptual, visualizar mensajes en las distintas celdas de la pantalla y ajustar el contraste, buscando la configuración en la que el contenido resultara más legible.

```

1     #include <Wire.h>
2     #include <LiquidCrystal_I2C.h>
3     #include <Arduino.h>
4
5     LiquidCrystal_I2C lcd(0x27, 20, 4); // (direccion, columnas, filas)
6
7     void setup() {
8         Wire.begin(21, 22); // Pines I2C en ESP32 (SDA, SCL)
9         lcd.init();         // Inicializa el LCD
10        lcd.backlight();     // Enciende la luz de fondo
11        lcd.clear();
12
13        lcd.setCursor(0, 0); // columna 0, fila 0
14        lcd.print("Fila 1 funcionando");
15        lcd.setCursor(0, 1);
16        lcd.print("Fila 2 funcionando");
17        lcd.setCursor(0, 2);

```



```

18     lcd.print("Fila 3 funcionando");
19     lcd.setCursor(0, 3);
20     lcd.print("Fila 4 funcionando");
21 }
22
23 void loop() {
24 }

```

Estas pruebas permitieron ajustar parámetros eléctricos, de comunicación y de software antes de pasar a la integración completa.

Prototipos e integración del sistema Con los módulos validados, se avanzó a la construcción de prototipos integrados:

- Diseño del **sistema de abastecimiento hidráulico con control de nivel**, definiendo las interconexiones entre tanque, bomba y sensores para evitar usos inseguros, una vez montado realizamos la calibración del sistema, garantizando una buena precisión en sus funciones.
- Desarrollo de la **lógica del sistema** tanto en el microcontrolador como en la plataforma ThingsBoard, de forma que ambos trabajen coordinados.
- Implementación de un **módulo de interacción y validación de clientes**, encargado de procesar la petición de carga y mostrar la información en el display.
- Configuración de **dashboards diferenciados** para la empresa y para el usuario final, permitiendo gestión de usuarios, seguimiento de cargas y visualización de alarmas.
- Definición de la **comunicación entre sensores y actuadores** y del esquema de **alimentación eléctrica**, priorizando seguridad, simplicidad y facilidad de mantenimiento.
- Diseño del **modelo físico y estructura del surtidor**, considerando materiales y disposición de componentes para un montaje robusto.

En conjunto, este plan de trabajo permitió pasar de las pruebas individuales de cada componente a un prototipo integrado de Saabaco, capaz de realizar el abastecimiento, registrar a los usuarios y reportar la información en tiempo real a la nube.

Esto se ve representado en los siguientes diagramas de Gantt, que describen el procedimiento seguido a lo largo del proyecto. Cabe aclarar que se trata de aproximaciones: estos diagramas se utilizaron como referencia durante el desarrollo y la duración de cada etapa fue ajustándose según las dificultades encontradas en cada una de ellas. Sin embargo, en líneas generales, constituyen una buena descripción del desarrollo real del proyecto.

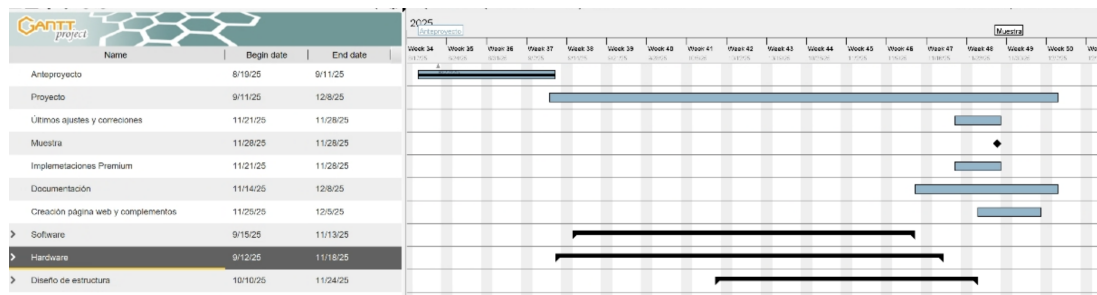


Figura 5.1: Gantt1

Si desplegamos las secciones de este Gannt se llega a el siguiente donde se pueden apreciar mejor las partes mas detalladamente.

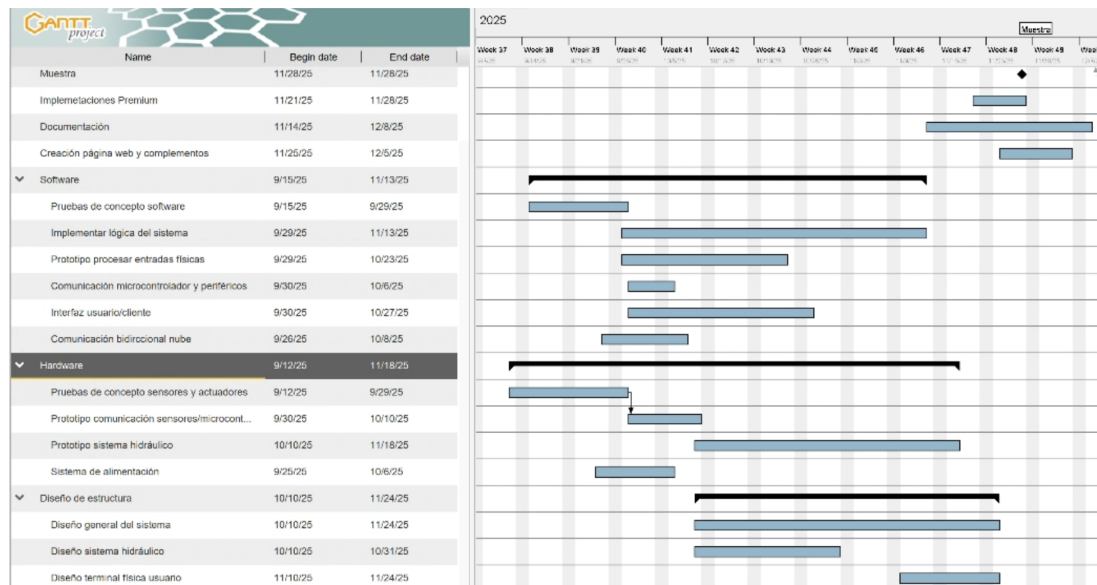


Figura 5.2: Gantt2

Capítulo 6

Bitácora

6.0.1. Martes 18/8

- Se introdujo el proyecto
- se propusieron ideas.

6.0.2. Jueves 21/8

- Presentacion de una idea sólida, con un punteo de features más avanzado.

6.0.3. Martes 26/8

- Se presentaron los features en tres niveles de complejidad.
- Los docentes ayudaron a definir la lista final de features.

6.0.4. Jueves 28/8

- Presentación de metodologías ágiles.
- Se adelanto el anteproyecto.

6.0.5. Martes 2/9

- Presentación de ThingsBoard, plataforma IoT.
- Se continuo el anteproyecto.

6.0.6. Jueves 4/9

- Se avanzó la documentación del anteproyecto.

6.0.7. Martes 9/9

- Presentaciones PPT y entrega anteproyecto.
- Entrega anteproyecto.

6.0.8. Martes 16/9

- Se recibieron compras.
- Vimos PPT de Git.
- Investigamos ThingsBoard y chequeamos que las compras recibidas funcionen, a excepción de la ESP32.

6.0.9. Jueves 18/9

- Se realizo el cuestionario.
- Se encontro una falla de la ESP32 en su hardware.
- Prueba de concepto sensor de calidad de aire fallida, se pensaron formas de testear el sensor.
- Prueba de concepto widget nivel de tanque.

6.0.10. Martes 23/9

- Se realizaron pruebas de concepto de los sensores de Distancia SRF05, .
- Prueba de concepto módulo RFID, se logro identificar y diferenciar una tarjeta y un llavero.
- Prueba de concepto pantalla LCD, utilizando el módulo adaptador LCD a I2C ya incorporado a la pantalla logrando mostrar mensajes estáticos en todas sus filas.
- Se investigo sobre el sensor MQ-135 encontrando el problema en su funcionamiento.

6.0.11. Viernes 26/9

- Tras dejar 48 horas calentando el sensor MQ-135, se volvio a intentar la prueba de concepto, almacenando humo en una botella para activar el sensor. El resultado no fue el esperado, el sensor logro identificar notoriamente la presencia de humo, pero marcando pequeños voltajes y un cambio en el valor adc notorio pero bajo. Como positivo se encontro que no genera ruido, el valor sin humo es adc=0 fijo.

6.0.12. Martes 30/9

- Se trabajo con visual studio code, configurandolo para Arduino, facilitando el uso de GitHub y reduciendo tiempos de compilación.
- Se recibio la bomba.
- Se trabajo con la plataforma thingsboard para familiarizarse mas con los activos, dispositivos y cadenas de reglas
- Hubo una devolucion de el anteproyecto, comentando pros y contras.

6.0.13. Jueves 2/10

- Se incorpore el sensor de distancia al código IoT, logrando enviar y recibir en la nube datos en formato JSON.

6.0.14. Martes 7/10

- Prueba de concepto, se logro comunicar y dejar funcionando el tanque con los datos recibidos de la Esp.
- Prueba de concepto, se investigo sobre el uso de un módulo MOSFET, generamos una señal PWM desde la ESP para generar una variación PWM en los 12V destinados a la bomba.

6.0.15. Jueves 9/10

- Se logro implementar en thingsBoard el registro de usuario
- Se realizo un diagrama de estados de el sistema general

6.0.16. Martes 21/10

- Se comenzo la implementación del codigo de la placa secundaria.
- Se realizo la logica de thingsboard para los usuarios

6.0.17. Jueves 23/10

- Avanzamos con el código de ambas placas.
- Se corrigieron problemas de thingsboard y se culmino la parte de pruebas en thingsboard para comenzar a hacer el dashboard final

6.0.18. Martes 28/10

- Probamos el caudalimetro y analizamos posibles circuitos para mejorar su señal.
- Se implemento la comunicacion con la placa y con la api.

6.0.19. Jueves 30/10

- Avanzamos con el código de la placa principal.
- Se hizo el dashboard de usuarios implementando en cadena de reglas la logica necesaria.

6.0.20. Martes 4/11

- Se comenzo a probar el codigo de la placa principal, se realizaron ajustes y se encontro un pin defectuoso.

6.0.21. Jueves 6/11

- Se probaron ambas placas logrando un funcionamiento parcial, se identificaron mejoras a realizar.

6.0.22. Martes 11/11

- Se probo el sistema general integrado por completo, idenficiando errores en thingsboard y en la placa para luego arreglarlos.

6.0.23. Jueves 13/11

- Se continuaron identificando errores a mejorar.
- Se integro el diseño del sistema con los componentes del proyecto.

6.0.24. Martes 18/11

- Una vez culminada el periodo de pruebas se acomodo el diseño para comodidad y alcance de los cables.
- Se realizaron mediciones de algunos componentes para culminar la plataforma IoT.

6.0.25. Jueves 20/11

- Se desarmo todo el sistema para soldar los cables.

6.0.26. Martes 25/11

- Se soldaron los cables.

6.0.27. Jueves 27/11

- Se soldaron cables.
- Se implemento el tanque al sistema fisico.

Apéndice A

Códigos de ambas placas

A.1. Código de la placa principal (ESP32)

```
1 #include <Arduino.h>
2 #include <WiFi.h>
3 #include <PubSubClient.h>
4 #include <ArduinoJson.h>
5 #include <LiquidCrystal_I2C.h>
6 #include <Wire.h>
7 #include <SPI.h>
8 #include <MFRC522.h>
9
10
11
12
13 //Variables WiFi
14 const char* ssid = "HUAWEI-IoT";
15 const char* password = "ORTWiFiIoT";
16 //const char* ssid = "canelita";
17 //const char* password = "Mostro1515";
18 const char* mqtt_server = "demo.thingsboard.io";
19 const char* token = "XigmXofElTIqmQUQHhJB";
20 // Connection objects
21 WiFiClient espClient;
22 PubSubClient client(espClient);
23
24 //timestamp of the last telemetry update
25 unsigned long lastMsg = 0;
26 unsigned long lastMsg3 = 0;
27
28 //buffer sizes and messages
29 #define MSG_BUFFER_SIZE (50)
30 char msg[MSG_BUFFER_SIZE];
31 char msg2[MSG_BUFFER_SIZE];
32
33 //json document to store incoming messages
34 DynamicJsonDocument incoming_message(256);
35
```

```

36      //Variables Fsm
37  int fsm = 0;
38  int estado_habilitacion = 0; // 0 no establecido, 1 habilitado, -1 no
    habilitado
39  float precio = 0; // Precio a mostrar en LCD
40  bool alarma = true; // Alarma activa
41  int cantidad = 0;
42
43      //Asignacion de pines
44  const int trigPin = 32; // Pin Trigger
45  const int echoPin = 14; // Pin Echo
46  const int sensorHumo = 34; // Pin sensor de humo
47  const int enablePwm = 25; // Pin de habilitacion de PWM
48  const int finBombeo = 39; // Pin de fin de bombeo
49  const int SS_PIN = 5; //SDA
50  const int RST_PIN = 26 ; // Reset
51
52      //Variables sensor ultrasonico
53  long duracion;
54  long distancia;
55
56  // Variables RFID
57  String uidString = "";
58
59      // Configuracion LCD
60  LiquidCrystal_I2C lcd(0x27, 20, 4);
61  String linea0 = "";
62  String linea1 = "";
63  String linea2 = "";
64  String linea3 = "";
65
66  String msgCantidad = "  Ingrese cantidad";
67  String msgId = "  e identifiquese:";
68  String mensajeCant = "";
69  String mensajePrecio = "";
70
71  bool conteoActivo = false;
72  unsigned long lastMsg2 = 0;
73
74  //RFID
75  //Parameters
76  const int ipaddress[4] = {103, 97, 67, 25};
77
78  //Variables
79  byte nuidPICC[4] = {0, 0, 0, 0};
80  MFRC522 rfid = MFRC522(SS_PIN, RST_PIN);
81
82
83  //Initiates WiFi Connection
84  void setup_wifi() {
85

```



```

86   delay(10);
87   // We start by connecting to a WiFi network
88   Serial.println();
89   Serial.print("Connecting to ");
90   Serial.println(ssid);
91
92   WiFi.mode(WIFI_STA);
93   WiFi.begin(ssid, password);
94
95   while (WiFi.status() != WL_CONNECTED) {
96       delay(500);
97       Serial.print(".");
98   }
99
100  randomSeed(micros());
101
102  Serial.println("");
103  Serial.println("WiFi connected");
104  Serial.println("IP address: ");
105  Serial.println(WiFi.localIP());
106 }
107
108 //Gestiona mensajes entrantes RPC
109 void callback(char* topic, byte* payload, unsigned int length) {
110
111     //log to console
112     Serial.print("Message arrived [");
113     Serial.print(topic);
114     Serial.print("] ");
115     for (int i = 0; i < length; i++) {
116         Serial.print((char)payload[i]);
117     }
118     Serial.println();
119
120     //convert topic to string to parse
121     String _topic = String(topic);
122
123     if (_topic.startsWith("v1/devices/me/rpc/request/")) { //Aca se manejan las
        Request de thingsboard
124         //We are in a request, check request number
125         String _number = _topic.substring(26);
126
127         //Read JSON Object
128         deserializeJson(incoming_message, payload);
129         String metodo = incoming_message["method"];
130         JsonVariant paramsAux = incoming_message["params"];
131         String params = paramsAux.as<String>();
132
133
134         if (metodo=="permisos_usuario" && fsm == 2) {
135

```

```

136     if (params == "true")
137     {
138         estado_habilitacion = 1;
139         Serial.println("Dispositivo habilitado para operar.");
140     }
141     else
142     {
143         estado_habilitacion = -1;
144         Serial.println("Dispositivo NO habilitado para operar.");
145     }
146
147
148 }
149
150 if (metodo=="valor_Nafta")
151 {
152     precio = params.toFloat();
153     Serial.println(precio);
154 }
155
156
157 if (metodo=="alarma_OK" && fsm == 4) // Señal apagar alarma dado que esta
encendida
158 {
159     alarma = false;
160 }
161
162
163 }
164 }
165
166 //Reconecta conexion MQTT
167 void reconnect() {
168     // Loop until we're reconnected
169     while (!client.connected()) {
170         Serial.print("Attempting MQTT connection...");
171         // Attempt to connect - client.connect(DEVICE_ID, TOKEN, TOKEN)
172         if (client.connect("NODEMCU Nuevo", token, token )) {
173             Serial.println("connected");
174             // Once connected, subscribe to rpc topic
175             client.subscribe("v1/devices/me/rpc/request/+");
176         } else {
177             Serial.print("failed, rc=");
178             Serial.print(client.state());
179             Serial.println(" try again in 5 seconds");
180             // Wait 5 seconds before retrying
181             delay(5000);
182         }
183     }
184 }
185

```

```

186 //Lectura analogica de sensor de humo
187 bool alarmaIncendio()
188 {
189     if (analogRead(sensorHumo) > 100) // Umbral de deteccion
190     {
191         Serial.println(sensorHumo);
192         return true;
193     }
194     return false;
195 }
196
197 //Limpia una linea del LCD , 0 a 3
198 void clearLine(unsigned int row) {
199     lcd.setCursor(0, row);
200     lcd.print("                "); // 20 espacios para un LCD 20x4
201     lcd.setCursor(0, row); // opcional: volver al inicio de la línea
202 }
203
204 //Limpia todo el display sin retardo bloqueante
205 void clearDisplay() {
206     for (int i = 0; i < 4; i++) {
207         clearLine(i);
208     }
209 }
210
211 //Recibe 4 strings y los muestra en el LCD
212 void mostrarLCD(String m0, String m1, String m2, String m3)
213 {
214     clearDisplay();
215     lcd.setCursor(0, 0); // columna 0, fila 0
216     lcd.print(m0);
217     lcd.setCursor(0, 1);
218     lcd.print(m1);
219     lcd.setCursor(0, 2);
220     lcd.print(m2);
221     lcd.setCursor(0, 3);
222     lcd.print(m3);
223 }
224
225 //Envia un mensaje por telemetria con topico y value
226 void enviarTelemtria(String topico, String value)
227 {
228     //Send value as telemetry
229     DynamicJsonDocument resp(256);
230     resp[topico] = value;
231     char buffer[256];
232     serializeJson(resp, buffer);
233
234     client.publish("v1/devices/me/telemetry", buffer);
235
236     Serial.print("Publish message [telemetry]: ");

```

```

237     Serial.println(buffer);
238 }
239
240 //Toma una medida del sensor ultrasonico
241 int SensorDistancia()
242 {
243     // 1. Enviar un pulso de 10 microsegundos al pin Trigger para iniciar la
        medición
244     digitalWrite(trigPin, LOW);
245     delayMicroseconds(2); // Un pequeño delay antes de enviar el pulso alto
246     digitalWrite(trigPin, HIGH);
247     delayMicroseconds(10);
248     digitalWrite(trigPin, LOW);
249
250     // 2. Medir el tiempo de retorno del pulso en el pin Echo
251     // La función pulseIn() retorna el tiempo que el pin estuvo en estado HIGH
252     long duracion = pulseIn(echoPin, HIGH);
253
254     // 3. Calcular la distancia en centímetros
255
256     int distancia = duracion / 29 / 2;
257     distancia = int(distancia);
258
259     // 4. Mostrar la distancia en el Monitor Serial
260     Serial.print("Distancia: ");
261     Serial.print(distancia);
262     Serial.println(" cm");
263
264     return distancia;
265 }
266
267
268 // intenta leer el RFID y devuelve true si se leyó uno nuevo
269 bool readRFID() {
270     if (!rfid.PICC_IsNewCardPresent()) return false; // no hay tarjeta
271     if (!rfid.PICC_ReadCardSerial()) return false; // no se pudo leer UID
272
273
274     Serial.print(F("UID: "));
275     uidString = "";
276     for (byte i = 0; i < rfid.uid.size; i++) {
277         if (rfid.uid.uidByte[i] < 0x10) uidString += "0";
278         uidString += String(rfid.uid.uidByte[i], HEX);
279     }
280     uidString.toUpperCase();
281
282     Serial.println("UID detectado: " + uidString);
283
284     rfid.PICC_HaltA();
285     return true;
286 }

```

```

287
288 void setup(){
289     Serial.begin(115200);
290
291     setup_wifi();
292     client.setServer(mqtt_server, 1883);
293     client.setCallback(callback);
294
295     // LCD
296     Wire.begin(21, 22); // Pines I2C en ESP32 (SDA, SCL)
297     delay(100);
298     lcd.init();          // Inicializa el LCD
299     lcd.backlight();     // Enciende la luz de fondo
300     lcd.clear();
301     lcd.setCursor(0, 0); // columna 0, fila 0
302     lcd.print("Inicio");
303
304     // Pines sensor Distancia
305     pinMode(trigPin, OUTPUT);
306     pinMode(echoPin, INPUT);
307     pinMode(enablePwm, OUTPUT);
308     pinMode(finBombeo, INPUT);
309
310     digitalWrite(enablePwm, LOW); // Deshabilito PWM al inicio
311
312
313     //Sensor humo
314     analogReadResolution(12); // Lectura de 0-4095
315
316     //RFID
317     SPI.begin();
318     rfid.PCD_Init();
319
320     Serial.print(F("Reader :"));
321     rfid.PCD_DumpVersionToSerial();
322 }
323
324
325
326 void loop()
327 {
328     //Codigo que siempre se ejecuta
329     if (!client.connected())
330     {
331         reconnect();
332     }
333     client.loop(); // trigger de callbacks
334
335     unsigned long now = millis();
336     if (now - lastMsg > 10000) // Actualzo cada 10 segundos
337     {

```

```

338     lastMsg = now;
339     enviarTelemtria("distancia_cm", String(SensorDistancia()));
340
341
342 }
343 unsigned long now2 = millis();
344 if (now2 - lastMsg3 > 2000) {// Lectura de sensor cada 2 segundos
345     lastMsg3 = now2;
346
347     if(alarmaIncendio())
348     {
349         fsm = 4;
350     }
351 }
352
353
354
355 switch(fsm)
356 {
357     case 0:
358         // Reinicio del sistema
359         estado_habilitacion = 0;
360         cantidad = 0;
361         alarma = true;
362         fsm = 1;
363         break;
364
365     case 1:
366
367
368         // Leer UART por cantidad
369         if(Serial.available() )
370         {
371             String recibirMensaje = Serial.readStringUntil('\n');
372             cantidad = recibirMensaje.toInt();
373         }
374
375         // Mostrar en LCD
376         mensajeCant = "          " +String(cantidad);
377         mensajePrecio = "          $" + String(precio);
378
379         static unsigned long lastLCD = 0;
380         if (millis() - lastLCD > 100) { // cada 0.1 s
381             clearDisplay();
382             mostrarLCD(msgCantidad, msgId , mensajeCant, mensajePrecio);
383             lastLCD = millis();
384         }
385
386
387         // Si hay una nueva lectura RFID, enviar telemetrias y cambiar a
388         estado 2

```

```

388         if(readRFID())
389         {
390             enviarTelemtria("id_tarjeta", uidString);
391             enviarTelemtria("cantidad_carga", String(cantidad));
392             fsm = 2;
393         }
394
395         break;
396
397     case 2:
398         // Espero RPC
399         // Si es 1(true), envio señal de control PWM actualizo LCD y cambio
a estado 3
400         // Si es -1(false), nvio mensaje LCD y vuelvo a estado 1
401         // Si es 0 es indeterminado
402         if(estado_habilitacion != 0)
403         {
404             if(estado_habilitacion == 1)
405             {
406                 // Habilitado
407                 digitalWrite(enablePwm, HIGH); // Habilito PWM
408                 fsm = 3;
409                 clearDisplay();
410                 mostrarLCD("", "Despachando...", "", "");
411                 break;
412             }
413             else
414             {
415                 // No habilitado
416                 clearDisplay();
417                 mostrarLCD("", "Usuario no", "habilitado", "");
418                 estado_habilitacion = 0;
419
420                 if (!conteoActivo)
421                 {
422                     lastMsg2 = millis();
423                     conteoActivo = true;
424                 }
425
426                 unsigned long now2 = millis();
427                 if (now2 - lastMsg2 > 2000) // Muestro mensaje por 2 segundos
428                 {
429                     fsm = 1;
430                     conteoActivo = false;
431                     //Considerar posibles resets de variables
432                     cantidad = 0;
433                 }
434                 break;
435             }
436         }
437

```

```

438         break;
439
440     case 3:
441         // Leo señal de control recibida
442
443         if(digitalRead(finBombeo) == HIGH) // Fin de bombeo
444         {
445             clearDisplay();
446             mostrarLCD("", "Fin de bombeo", "", "");
447             estado_habilitacion = 0;
448             digitalWrite(enablePwm, LOW); // Apago PWM
449
450             if (!conteoActivo)
451             {
452                 lastMsg2 = millis();
453                 conteoActivo = true;
454             }
455
456             unsigned long now2 = millis();
457             if (now2 - lastMsg2 > 2000) // Muestro mensaje por 2 segundos
458             {
459                 fsm = 1;
460                 conteoActivo = false;
461             }
462         }
463         // HIGH, LCD fin, señal control PWM LOW y state 1
464         // LOW, mantengo estado
465         break;
466
467     case 4:
468         // Alarma de incendio
469         enviarTelemtria("alarmaActiva", "true");
470         delay(1000); // Evitamos bombardear con telemetria
471         // Actualizo LCD
472         clearDisplay();
473         mostrarLCD("!!!ALARMA DE", "INCENDIO ACTIVADA!!!", "Apague la alarma"
474         , "desde la app");
475
476         // Bloqueo PWM
477         digitalWrite(enablePwm, LOW);
478
479         // Implementar como volver a normalidad
480         if (!alarma) // Se desactivo la alarma
481         {
482             fsm = 0;
483         }
484         // El delay de este bloque esta implicito a recibir el RPC en t > 20
485         ms
486         break;
487     }

```


487
488

}

A.2. Código de la placa secundaria (ESP8266)

```
1  #include <Arduino.h>
2
3  int fsm ; //Indice de estados
4  const int flagBomba = 14; // Señal de control bombeo autorizado D5
5  const int flagFin = 12; // Señal de para indicar que termino el bombeo D6
6  const int btn1 = 16; // Boton incrementar D0
7  const int btn2 = 5; // Boton decrementar D1
8  const int pwm = 4; // Salida pwm D2
9  const int flujo = 13; // Entrada sensor flujo D7
10
11
12  String mandarMensaje;
13  int cuenta = 0; // Valor de seleccion por botones
14  int valorVol = 0; // Valor recibido y leído una vez de volumen a dispensar
15
16  //PWM:
17  const unsigned long medioPeriodo = 10; // ms → 50 Hz (T=20 ms)
18  unsigned long anterior = 0;
19  bool estado = false;
20
21  //Sensor Flujo
22  bool risingEdge = false;
23  unsigned long contador1 = 0;
24  long acumulado = 0;
25  int pulsos = 0;
26  const float k = 7.1; // Constante de calibracion del sensor de flujo
27
28  void setup() {
29
30      Serial.begin(115200);
31      fsm = 0;
32      pinMode(flagBomba, INPUT);
33      pinMode(flagFin, OUTPUT);
34      pinMode(btn1, INPUT);
35      pinMode(btn2, INPUT);
36      pinMode(pwm, OUTPUT);
37      pinMode(flujo, INPUT);
38      digitalWrite(pwm, LOW);
39      digitalWrite(flagFin , LOW);
40
41  }
42
43
44  void loop() {
45
```

```

46  switch (fsm) {
47
48      case 0: // Estado de inicio
49
50          fsm = 1;
51          break;
52
53      case 1: // Estado lectura de botones
54          digitalWrite(flagFin , LOW);
55          // Llego la señal para iniciar bombeo por el pin de control
56
57          if (digitalRead(flagBomba) == HIGH)
58          {
59              fsm = 2;
60              valorVol = cuenta;
61              cuenta = 0;
62              break;
63          }
64
65          if (digitalRead(btn1) == HIGH && cuenta < 2000)
66          {
67              cuenta = cuenta + 10;
68              delay(200);
69              mandarMensaje = String(cuenta);
70              Serial.println(mandarMensaje);
71
72
73          }
74          if (digitalRead(btn2) == HIGH && cuenta > 0)
75          {
76              cuenta = cuenta - 10;
77              delay(200);
78              mandarMensaje = String(cuenta);
79              Serial.println(mandarMensaje);
80
81
82          }
83
84
85
86
87      break;
88
89
90      case 2: // Estado control bomba
91
92          if (valorVol >= 5 && digitalRead(flagBomba) == HIGH) // valor ya
decrementado
93          {
94              // Generacion de señal PWM
95              unsigned long ahora = millis();

```

```

96     if (ahora - anterior >= medioPeriodo)
97     {
98         if (estado) {
99             digitalWrite(pwm, HIGH);
100         } else {
101             digitalWrite(pwm, LOW);
102         }
103         estado = !estado;
104         anterior = ahora;
105         break;
106     }
107
108     if (pulsos < 4)
109     {
110         if (digitalRead(flujo) == HIGH && !risingEdge)
111         {
112
113             risingEdge = true;
114             if (pulsos == 0)
115             {
116                 contador1 = millis();
117             }
118
119             unsigned long ahora = millis();
120             acumulado = acumulado + ahora - contador1;
121             pulsos = pulsos + 1;
122             contador1 = ahora;
123         }
124
125         if (digitalRead(flujo) == LOW && risingEdge)
126         {
127             risingEdge = false;
128         }
129
130         break;
131     }
132     else
133     {
134         float tiempoT = acumulado*0.001; // Escalado a segundos
135         float periodo = tiempoT / 3;
136         float caudal = 1.0 / (periodo*k); // Litros por segundo
137         valorVol = valorVol - tiempoT*caudal; // Volumen en litros
138
139         acumulado = 0;
140         pulsos = 0;
141         risingEdge = true; // Ignoro este pulso
142     }
143 }
144 else
145 {
146     digitalWrite(flagFin , HIGH); // Indico que el bombeo ha finalizado

```

```

147     mandarMensaje = String(cuenta);
148     Serial.println(mandarMensaje);
149
150
151     acumulado = 0;
152     pulsos = 0;
153     valorVol = 0;
154     fsm = 1;
155     delay(200); // La placa principal tiene un delay no bloqueante mayor
a 20 ms, este delay permite sincronizar la flagBomba
156     break;
157 }
158 break;
159 }
160 }

```